开源模型后训练与 VPS 部署实战教程

以单张 RTX 4070 12GB 为主要硬件边界：CPT → SFT → DPO，完成 QLoRA 后双轨部署——GPU 用 vLLM，CPU 用 GGUF + Ollama / llama.cpp。

01 · CPT	02 · SFT	03 · DPO	04 · Serve
让模型吸收领域语料的统计分布	把知识变成可调用的指令行为	用成对偏好塑造回答边界	GPU vLLM / CPU GGUF 双轨服务

适合：已有 Python / PyTorch 基础，手头是 12GB 消费级显卡，希望真正跑通训练、量化和自托管服务的开发者。

先给结论：12GB 显卡决定了两条部署路线

RTX 4070 的 12GB 显存足够用 QLoRA 微调 7B 级模型，但不适合 7B 全参数训练，也装不下 7B 的 fp16 / bf16 权重再加 KV cache。训练与上线都要主动量化。

GPU 主机：RTX 4070 12GB

- QLoRA：7B、4-bit NF4、batch=1
- vLLM：7B AWQ / GPTQ 4-bit
- 或 3B 以下 bf16，留空间给 KV cache
- 适合更低延迟与多请求连续批处理
- API 仍只绑定本机，再经 SSH / TLS 访问

CPU VPS：长期、低成本

- 最终权重转换为 GGUF
- llama.cpp：参数透明、资源可控
- Ollama：模型导入和服务管理更简单
- 默认 Q4_K_M，按 RAM 决定 Q5 / Q8
- 适合低并发、成本优先、GPU 不常在线

第一个可复现闭环

环节	12GB 起步配置	边界
训练	7B QLoRA；seq 1024；batch 1；累积 16	先冒烟，再逐步加长度
GPU 推理	7B GPTQ / AWQ 4-bit；ctx 2K–4K	7B fp16 会 OOM
CPU 推理	7B GGUF Q4_K_M；ctx 2K；并发 1	看 VPS RAM 与内存带宽
验收	同一评测集比较合并权重、GPU 量化、GGUF	不把格式差异误判为模型退化

硬边界 12GB 下，7B QLoRA 可行；7B 全参数微调不可行；vLLM 直接加载 7B fp16 / bf16 也不可行。量化不是“可选优化”，而是容量设计。

四个阶段解决四类不同问题

CPT

输入是原始领域文本。目标仍是“预测下一个 token”。主要改变知识分布、术语与文体。

SFT

输入是高质量对话或指令-答案。学习何时回答、怎么组织、如何遵循格式。

DPO

输入是同一 prompt 下的 chosen / rejected。学习偏好差异，不负责灌输大批新知识。

Deploy

把训练产物合并、量化、服务化。优化的是内存、延迟、并发、鉴权和回滚。

不要用错工具

你观察到的问题	首选手段	不该先做
不知道公司术语、产品手册	CPT 或 RAG	仅靠 DPO
知道事实但回答格式很差	SFT	继续堆原始文本
经常过度解释、拒答风格不合适	DPO	盲目增加 SFT 轮次
VPS 内存不够	更小模型、GGUF 量化、缩短上下文	修改训练数据

核心原理

LoRA

冻结原权重 W，只训练低秩增量 ΔW = B · A。可训练参数和 checkpoint 都更小。

QLoRA

把冻结的基座以 4-bit NF4 载入，LoRA 参数仍用较高精度计算。省显存，不等于“最终部署量化”。

GGUF

部署格式。Q4_K_M 等量化发生在训练完成后，目标是降低 VPS 内存和 CPU 推理成本。

关键区别 QLoRA 是训练时的显存策略；GGUF 是部署时的权重格式。两者不是同一个步骤。

显存—模型规模对照：以 12GB 为基准

先算权重，再给 KV cache、CUDA kernel、临时缓冲和容器留空间。bf16 / fp16 权重大约每参数 2 字节；4-bit 权重理论下限约每参数 0.5 字节，但量化元数据和运行时会额外占用显存。

显存	训练建议	vLLM 推理建议	不要做
8GB	3B QLoRA；7B 仅极限短序列实验	3B 4-bit；小上下文	7B fp16
12GB · RTX 4070	**7B QLoRA：NF4、batch 1、seq 512–1024、检查点**	**7B AWQ/GPTQ 4-bit；或 ≤3B bf16**	**7B 全参训练；7B fp16/bf16 vLLM**
16GB	7B QLoRA 更宽松，可试 2K	7B 4-bit，可提高 ctx / 并发	仍不建议 7B 全参
24GB	7B QLoRA 较宽松；可探索 14B QLoRA	7B bf16 或 14B 4-bit	忽略 KV cache 直接按权重塞满

12GB 的默认答案

- 训练：7B + NF4 + double quant
- LoRA rank 16；batch 1
- gradient checkpointing 开启
- seq 1024，累积 16
- DPO 更紧：先降到 seq 512

推理量化怎么选

- vLLM：先用模型已发布的 AWQ / GPTQ
- CPU：GGUF Q4_K_M 默认
- ≤3B 可用 bf16，但仍从 4K ctx 起步
- 量化格式必须被当前 vLLM / GPU 支持
- 增大 ctx 和并发前记录峰值 VRAM

为什么 7B fp16 不行 7B 仅权重就约 14GB（十进制估算），已经超过 12GB；vLLM 还要 KV cache 和运行时空间。不要靠把 gpu-memory-utilization 设到 1.0 硬塞。

RTX 4070 12GB：7B QLoRA 配置矩阵

下面是单卡 12GB 的保守起点，不是所有 7B 架构的显存保证。先跑 20–50 step 并记录 `nvidia-smi` 峰值，再决定是否增加序列长度。

目标	max_length	micro batch	梯度累积	建议
最稳冒烟	512	1	16	先验证数据、模板、保存与合并
SFT 默认	**1024**	**1**	**16**	**有效 batch 16；rank 16；packing 开**
更长文本	2048	1	32	可能 OOM；先用 Flash Attention 兼容实现并实测
DPO 默认	512	1	16	chosen + rejected 更吃显存；prompt 256 起

必须同时打开的节省项

模型侧

- 4-bit NF4
- double quant
- LoRA rank 16
- 冻结基座，不保存全量梯度

训练侧

- gradient checkpointing
- `use_cache=False`
- batch=1，靠累积做有效 batch
- bf16 支持时优先 bf16 计算

OOM 降级顺序

1. 确认没有其它进程占显存；重启干净进程。
2. seq 1024 → 768 → 512；batch 保持 1。
3. LoRA rank 16 → 8；只训练注意力投影层。
4. DPO 仍 OOM 时先在 3B 走通，或换更大显存机器。

不做全参数微调 Adam 类全参数训练不仅要装 7B 权重，还要梯度、优化器状态和激活；12GB 远远不够。QLoRA 的“可行”来自冻结 4-bit 基座，只训练小型 LoRA。

环境与目录：先把实验变成可复现工程

训练机准备

```
sudo apt update
sudo apt install -y git git-lfs python3-venv build-essential
mkdir -p ~/llm-lab && cd ~/llm-lab
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -U pip wheel
pip install -U torch transformers datasets accelerate peft trl \
  bitsandbytes safetensors sentencepiece huggingface_hub
python - <<'PY'
import torch, transformers, datasets, peft, trl
print("cuda:", torch.cuda.is_available(), torch.cuda.get_device_name(0))
print("torch:", torch.__version__, "transformers:", transformers.__version__)
print("datasets:", datasets.__version__, "peft:", peft.__version__, "trl:", trl.__version__)
PY
pip freeze > requirements.lock.txt
```

先升级再冻结，是为了得到一组在同一次实验中一致的版本。以后复现使用 `pip install -r requirements.lock.txt`。不要把“latest”当作可复现版本。

建议目录

```
llm-lab/
├── data/{cpt,sft,dpo,eval}/
├── scripts/{train_cpt.py,train_sft.py,train_dpo.py,merge.py}/
├── runs/{cpt,sft,dpo}/
├── merged/{cpt,sft,dpo}/
├── gguf/
├── reports/
└── requirements.lock.txt
```

> **模型选择** CPT 从 Base 模型开始；若跳过 CPT，SFT 也建议从 Base 开始。不要把已有 Instruct 模型当“纯基座”而忽略其既有对齐。

数据合同：三种阶段，三种 JSONL

CPT：原始领域文本

```
{"text":"退款申请须在订单完成后 7 天内提交……"}
{"text":"设备告警 E104 表示冷却回路压力不足……"}
```

SFT：对话消息

```
{"messages":[
  {"role":"system","content":"你是设备运维助手。"},
  {"role":"user","content":"E104 如何排查？"},
  {"role":"assistant","content":"先停机并确认压力表……"}
]}
```

DPO：显式 prompt + chosen / rejected

```
{"prompt":[{"role":"user","content":"E104 如何排查？"}],
 "chosen":[{"role":"assistant","content":"先停机并执行锁定挂牌……"}],
 "rejected":[{"role":"assistant","content":"直接拆开冷却泵看看。"}]}
```

必须做

- 逐条确认来源与许可证
- 去除个人身份信息和密钥
- 近重复去重，避免评测泄漏
- 按文档 / 客户 / 时间切分 train 与 eval
- 保留原始数据、清洗脚本与哈希

质量抽检

- CPT：编码、乱码、重复、广告
- SFT：答案正确、可执行、风格一致
- DPO：两个答案只差在目标偏好
- 拒绝样本不能只是明显胡话
- 评测集训练期间不可回流

训练前门禁：让坏数据尽早失败

下面的最小校验器不替代语义审核，但能挡住最常见的字段缺失、空答案和 chosen/rejected 相同问题。

```
# scripts/validate_data.py
import json, sys
from pathlib import Path

mode, path = sys.argv[1], Path(sys.argv[2])
rows = [json.loads(x) for x in path.read_text(encoding="utf-8").splitlines() if x.strip()]
assert rows, "empty dataset"

def text_of(messages):
    assert isinstance(messages, list) and messages
    for m in messages:
        assert m.get("role") in {"system", "user", "assistant"}
        assert isinstance(m.get("content"), str) and m["content"].strip()
    return "
".join(m["content"].strip() for m in messages)

for i, row in enumerate(rows, 1):
    if mode == "cpt":
        assert isinstance(row.get("text"), str) and len(row["text"].strip()) >= 40
    elif mode == "sft":
        text_of(row["messages"])
        assert row["messages"][-1]["role"] == "assistant"
    elif mode == "dpo":
        text_of(row["prompt"]); c = text_of(row["chosen"]); r = text_of(row["rejected"])
        assert c != r, f"row {i}: identical pair"
    else:
        raise ValueError("mode must be cpt/sft/dpo")
print(f"OK: {len(rows)} rows in {path}")
```

```
python scripts/validate_data.py cpt data/cpt/train.jsonl
python scripts/validate_data.py sft data/sft/train.jsonl
python scripts/validate_data.py dpo data/dpo/train.jsonl
```

切分原则

如果同一手册的相邻段落同时落入训练集和评测集，评测会虚高。应先按来源文档 ID 分组，再整体切分；客服数据则按会话 ID 或客户 ID 切分。

> **最低样本策略** 先用 100 条 CPT 文本、50 条 SFT、30 对 DPO 做 20–50 step 冒烟测试。确认 loss、保存、合并、生成都通，再跑全量。

CPT 原理：改变语料分布，而不是教聊天礼仪

继续预训练仍最小化因果语言模型损失：给定前文 token，预测下一个 token。它适合领域术语、知识密集语料和稳定文体。

为什么有效

模型在领域文本上重复看到术语共现、结构和事实关系，参数会向该领域分布移动。LoRA 只提供受限的低秩更新，所以成本低、回退容易。

为什么会失败

语料脏、学习率过高或训练过久会导致灾难性遗忘；重复数据会让模型背诵；短文档拼接不当会制造跨文档伪关联。

推荐起点（不是固定答案）

参数	起点	调节逻辑
LoRA rank	16 或 32	容量不足再增大
学习率	5e-5 至 1e-4	遗忘明显就降低
序列长度	1,024	文档结构重要再增大
通用语料混合	5%–20%	用通用评测决定比例
epoch	1 起步	以验证 loss 和任务指标早停

判断是否需要 CPT 如果知识经常变化，优先 RAG；如果术语、语言风格和稳定知识需要内化，再做 CPT。两者可以组合。

CPT 代码（1/2）：4-bit 基座 + LoRA

```
# scripts/train_cpt.py
import os, torch
from datasets import load_dataset
from transformers import (
    AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig,
    DataCollatorForLanguageModeling, Trainer, TrainingArguments
)
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training

MODEL = os.getenv("MODEL", "Qwen/Qwen3-0.6B-Base")
OUT = os.getenv("OUT", "runs/cpt")
MAX_LEN = int(os.getenv("MAX_LEN", "1024"))
dtype = torch.bfloat16 if torch.cuda.is_bf16_supported() else torch.float16

bnb = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_use_double_quant=True,
    bnb_4bit_compute_dtype=dtype,
)
tok = AutoTokenizer.from_pretrained(MODEL, use_fast=True)
if tok.pad_token is None: tok.pad_token = tok.eos_token
model = AutoModelForCausalLM.from_pretrained(
    MODEL, quantization_config=bnb, torch_dtype=dtype
)
model.config.use_cache = False
model = prepare_model_for_kbit_training(model)
lora = LoraConfig(
    task_type="CAUSAL_LM", r=16, lora_alpha=32,
    lora_dropout=0.05, bias="none",
    target_modules=["q_proj","k_proj","v_proj","o_proj",
                    "gate_proj","up_proj","down_proj"],
)
model = get_peft_model(model, lora)
model.print_trainable_parameters()
```

`target_modules` 适用于常见 Qwen/Llama 类命名；换架构前先打印 `model.named_modules()` 确认。训练时不要使用 `device_map="auto"` 把模型随意切分到多卡。

> **12GB 显存策略** 7B 从 batch=1、MAX_LEN=1024、累积 16、LoRA rank 16 起步。仍 OOM 时先降 MAX_LEN，再把 rank 降到 8；gradient checkpointing 必须开启。

CPT 代码（2/2）：分块、训练与断点

```
raw = load_dataset("json", data_files={
    "train":"data/cpt/train.jsonl", "eval":"data/cpt/eval.jsonl"
})

def tokenize(batch):
    return tok(batch["text"], add_special_tokens=False)
encoded = raw.map(tokenize, batched=True, remove_columns=raw["train"].column_names)

def group(batch):
    merged = {k: sum(batch[k], []) for k in batch}
    total = (len(merged["input_ids"]) // MAX_LEN) * MAX_LEN
    return {k: [v[i:i+MAX_LEN] for i in range(0, total, MAX_LEN)]
            for k, v in merged.items()}
lm = encoded.map(group, batched=True)

args = TrainingArguments(
    output_dir=OUT, num_train_epochs=1,
    per_device_train_batch_size=1, per_device_eval_batch_size=1,
    gradient_accumulation_steps=16,
    learning_rate=8e-5, warmup_ratio=0.03,
    lr_scheduler_type="cosine", weight_decay=0.01,
    logging_steps=10, eval_strategy="steps", eval_steps=100,
    save_strategy="steps", save_steps=100, save_total_limit=2,
    bf16=(dtype == torch.bfloat16), fp16=(dtype == torch.float16),
    gradient_checkpointing=True, report_to="none",
    seed=42, data_seed=42,
)
trainer = Trainer(
    model=model, args=args,
    train_dataset=lm["train"], eval_dataset=lm["eval"],
    data_collator=DataCollatorForLanguageModeling(tok, mlm=False),
)
trainer.train(resume_from_checkpoint=os.getenv("RESUME") or None)
trainer.save_model(f"{OUT}/adapter_final")
tok.save_pretrained(f"{OUT}/adapter_final")
```

```
MODEL=Qwen/Qwen3-0.6B-Base MAX_LEN=1024 \
  python scripts/train_cpt.py
# 断点恢复
RESUME=runs/cpt/checkpoint-500 python scripts/train_cpt.py
```

分块函数会丢弃每个 map 批次末尾不足 MAX_LEN 的 token。正式大语料可改成流式 packing，并在每篇文档末尾插入 EOS，避免无边界拼接。

阶段衔接：先合并，再开始下一阶段

最容易理解、最少隐藏状态的做法，是把每阶段 LoRA 合并成一个新的全精度基座。0.6B 教学模型可直接做；大模型合并可能需要较多 CPU RAM，届时可保留 adapter 链或用分片加载。

```
# scripts/merge.py
import argparse, torch
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import PeftModel

p = argparse.ArgumentParser()
p.add_argument("--base", required=True)
p.add_argument("--adapter", required=True)
p.add_argument("--out", required=True)
a = p.parse_args()

tok = AutoTokenizer.from_pretrained(a.adapter, use_fast=True)
base = AutoModelForCausalLM.from_pretrained(
    a.base, torch_dtype=torch.float16, device_map="cpu",
    low_cpu_mem_usage=True,
)
model = PeftModel.from_pretrained(base, a.adapter)
model = model.merge_and_unload()
model.save_pretrained(a.out, safe_serialization=True, max_shard_size="4GB")
tok.save_pretrained(a.out)
print("saved:", a.out)
```

```
python scripts/merge.py \
  --base Qwen/Qwen3-0.6B-Base \
  --adapter runs/cpt/adapter_final \
  --out merged/cpt
```

产物命名不要含糊

目录	含义	下一步
runs/cpt/adapter_final	只含 CPT LoRA 增量	可回滚、可比较
merged/cpt	Base + CPT 的完整权重	作为 SFT 的 base
runs/sft/adapter_final	相对 merged/cpt 的 SFT 增量	合并后做 DPO
merged/dpo	最终完整权重	转 GGUF

不可逆风险 合并后的模型仍要保留原 base 标识、adapter、训练配置和数据哈希。只保留最终文件，会失去审计与回滚能力。

SFT 原理：把“会续写”变成“会按协议回答”

监督微调把聊天模板渲染后的整段 token 当作序列，最小化正确答案的交叉熵。它主要教行为、格式和任务模式。

高质量 SFT 示例

- 问题真实、有覆盖面
- 答案正确、可执行、不过度冗长
- 系统指令稳定
- 格式约束真的被答案遵守
- 困难样本和拒答边界都覆盖

常见失败

- 模板与部署时不一致
- 模型只会复述训练措辞
- 答案中混入来源标注噪声
- 过多低质量合成数据
- 训练全量 token 时学会模仿用户文本

损失掩码的取舍

最稳妥的第一版是对完整会话计算 loss，确保任何带聊天模板的 tokenizer 都能运行。确认模板支持 assistant token mask 后，再启用仅 assistant loss，减少对用户 prompt 的拟合。

模板一致性 训练、离线评测、GGUF 导出和 llama-server 必须使用同一套 chat template。乱码或“模型自问自答”往往不是权重坏了，而是模板错位。

SFT 代码：沿用 QLoRA，但换成对话数据

```
# scripts/train_sft.py
import os, torch
from datasets import load_dataset
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training
from trl import SFTConfig, SFTTrainer

MODEL = os.getenv("MODEL", "merged/cpt")
OUT = os.getenv("OUT", "runs/sft")
MAX_LEN = int(os.getenv("MAX_LEN", "1024"))
ACCUM = int(os.getenv("ACCUM", "16"))
dtype = torch.bfloat16 if torch.cuda.is_bf16_supported() else torch.float16
bnb = BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type="nf4",
    bnb_4bit_use_double_quant=True, bnb_4bit_compute_dtype=dtype)
tok = AutoTokenizer.from_pretrained(MODEL, use_fast=True)
assert tok.chat_template, "tokenizer 缺少 chat_template；先显式选择并保存模板"
if tok.pad_token is None: tok.pad_token = tok.eos_token
model = AutoModelForCausalLM.from_pretrained(MODEL,
    quantization_config=bnb, torch_dtype=dtype)
model.config.use_cache = False
model = prepare_model_for_kbit_training(model)
model = get_peft_model(model, LoraConfig(
    task_type="CAUSAL_LM", r=16, lora_alpha=32, lora_dropout=0.05,
    bias="none", target_modules=["q_proj","k_proj","v_proj","o_proj",
      "gate_proj","up_proj","down_proj"]
))
ds = load_dataset("json", data_files={
    "train":"data/sft/train.jsonl", "eval":"data/sft/eval.jsonl"})
args = SFTConfig(output_dir=OUT, max_length=MAX_LEN, packing=True,
    num_train_epochs=2, per_device_train_batch_size=1,
    per_device_eval_batch_size=1, gradient_accumulation_steps=ACCUM,
    learning_rate=1e-4, warmup_ratio=0.03, lr_scheduler_type="cosine",
    logging_steps=10, eval_strategy="steps", eval_steps=100,
    save_steps=100, save_total_limit=2, gradient_checkpointing=True,
    bf16=(dtype==torch.bfloat16), fp16=(dtype==torch.float16),
    report_to="none", seed=42, data_seed=42)
trainer = SFTTrainer(model=model, args=args,
    train_dataset=ds["train"], eval_dataset=ds["eval"],
    processing_class=tok)
trainer.train(resume_from_checkpoint=os.getenv("RESUME") or None)
trainer.save_model(f"{OUT}/adapter_final")
tok.save_pretrained(f"{OUT}/adapter_final")
```

```
# 12GB RTX 4070: 7B SFT 默认档
MODEL=Qwen/Qwen2.5-Coder-7B MAX_LEN=1024 ACCUM=16   python scripts/train_sft.py
python scripts/merge.py --base Qwen/Qwen2.5-Coder-7B   --adapter runs/sft/adapter_final --out merged/sft
```

`Qwen/Qwen2.5-Coder-7B` 只是已公开的 7B Base 容量示例；非代码领域应替换成许可证与任务匹配的 Base 模型。若 TRL 参数名不同，优先看当前安装版本的 SFTConfig 文档与 `python -c "from trl import SFTConfig; help(SFTConfig)"`。

DPO 原理：学习“更喜欢 A，而不是 B”

直接偏好优化（DPO）比较同一 prompt 下 chosen 与 rejected 的相对概率，并用冻结参考策略约束模型不要偏离 SFT 基线太远。

直观目标

让策略模型对 chosen 的相对偏好高于 rejected，同时减去参考模型原本就有的偏好。β 控制偏离参考模型的惩罚强度：过小可能激进，过大可能学不到足够差异。

简化表达：loss = -log sigmoid(β × [(policy chosen - ref chosen) - (policy rejected - ref rejected)])

一对好偏好数据应该怎样

- **同一问题：**chosen 与 rejected 必须回答同一个 prompt。
- **差异可解释：**例如安全步骤完整、引用正确、格式合规，而不是 chosen 长很多。
- **拒绝答案仍有迷惑性：**如果 rejected 全是胡话，模型学不到细粒度偏好。
- **长度去偏：**避免标注者总选更长回答；混入“短而好”的 chosen。
- **保留理由：**额外存 reviewer_reason 供审计，但训练前移除非标准列也可以。

DPO、奖励模型、GRPO 怎么选

方法	数据 / 信号	适合个人项目吗
DPO	成对偏好	是，最短路径
Reward + PPO	偏好 → 奖励模型 → 在线采样	复杂，先不做
GRPO	可验证奖励或规则函数	数学 / 代码等可验证任务

顺序约束 DPO 应从通过评测的 SFT 模型开始。用 Base 直接 DPO，通常会同时承担“学会对话”和“学会偏好”两件事，难以诊断。

DPO 代码：参考策略由 SFT 基线提供

```
# scripts/train_dpo.py
import os, torch
from datasets import load_dataset
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training
from trl import DPOConfig, DPOTrainer

MODEL = os.getenv("MODEL", "merged/sft")
OUT = os.getenv("OUT", "runs/dpo")
MAX_LEN = int(os.getenv("MAX_LEN", "512"))
MAX_PROMPT = int(os.getenv("MAX_PROMPT", "256"))
dtype = torch.bfloat16 if torch.cuda.is_bf16_supported() else torch.float16
bnb = BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type="nf4",
    bnb_4bit_use_double_quant=True, bnb_4bit_compute_dtype=dtype)
tok = AutoTokenizer.from_pretrained(MODEL, use_fast=True)
if tok.pad_token is None: tok.pad_token = tok.eos_token
model = AutoModelForCausalLM.from_pretrained(MODEL,
    quantization_config=bnb, torch_dtype=dtype)
model.config.use_cache = False
model = prepare_model_for_kbit_training(model)
model = get_peft_model(model, LoraConfig(
    task_type="CAUSAL_LM", r=16, lora_alpha=32, lora_dropout=0.05,
    bias="none", target_modules=["q_proj","k_proj","v_proj","o_proj",
      "gate_proj","up_proj","down_proj"]
))
ds = load_dataset("json", data_files={
    "train":"data/dpo/train.jsonl", "eval":"data/dpo/eval.jsonl"})
args = DPOConfig(output_dir=OUT, beta=0.1,
    max_length=MAX_LEN, max_prompt_length=MAX_PROMPT,
    num_train_epochs=1, per_device_train_batch_size=1,
    per_device_eval_batch_size=1, gradient_accumulation_steps=16,
    learning_rate=5e-6, warmup_ratio=0.05,
    logging_steps=10, eval_strategy="steps", eval_steps=50,
    save_steps=50, save_total_limit=2, gradient_checkpointing=True,
    bf16=(dtype==torch.bfloat16), fp16=(dtype==torch.float16),
    report_to="none", seed=42, data_seed=42)
trainer = DPOTrainer(model=model, ref_model=None, args=args,
    train_dataset=ds["train"], eval_dataset=ds["eval"],
    processing_class=tok)
trainer.train()
trainer.save_model(f"{OUT}/adapter_final")
tok.save_pretrained(f"{OUT}/adapter_final")
```

12GB 跑 7B DPO 时从 `MAX_LEN=512 MAX_PROMPT=256` 起步。这里 model 是 PEFT 模型，`ref_model=None` 让 TRL 使用关闭 adapter 的基线作为参考路径。不同 TRL 版本行为可能变化，升级后先跑 10 step 并检查日志与显存。

评测设计：始终与上一阶段做成对比较

只看训练 loss 不足以决定上线。每阶段都要固定同一组 prompt、采样参数和随机种子，对 Base / CPT / SFT / DPO 横向比较。

维度	方法	通过条件示例
领域知识	保留题、事实核对、引用定位	不低于上一版；关键事实零错误
通用能力	固定通用问答与摘要集	无明显灾难性遗忘
指令遵循	JSON、长度、步骤、拒答	机器可解析率达到目标
偏好胜率	盲评 A/B；随机左右位置	DPO 优于 SFT，且无安全退化
记忆风险	训练片段前缀续写、敏感串扫描	不泄漏秘密或个人信息
部署性能	TTFT、tok/s、峰值 RAM、错误率	满足 VPS 的实际 SLA

避免自欺

- 评测 prompt 不参与训练，也不用于反复手调单条答案。
- 事实题用可核验参考答案；开放题至少双人盲评或明确 rubric。
- 报告均值之外保留失败样本，按错误类型分桶。
- 量化前后都评一次；GGUF 的退化必须被单独测量。

> **最小上线门禁** CPT 不破坏通用集；SFT 格式合规；DPO 偏好胜率提升；GGUF 与合并模型的关键答案一致；VPS 峰值内存有余量。

离线生成脚本：保存可审计的 JSONL

```
# scripts/generate_eval.py
import argparse, json, torch
from pathlib import Path
from transformers import AutoModelForCausalLM, AutoTokenizer

p = argparse.ArgumentParser()
p.add_argument("--model", required=True)
p.add_argument("--prompts", default="data/eval/prompts.jsonl")
p.add_argument("--out", required=True)
a = p.parse_args()
tok = AutoTokenizer.from_pretrained(a.model)
model = AutoModelForCausalLM.from_pretrained(
    a.model, torch_dtype="auto", device_map="auto").eval()

with Path(a.out).open("w", encoding="utf-8") as f:
    for line in Path(a.prompts).read_text(encoding="utf-8").splitlines():
        row = json.loads(line)
        messages = row["messages"]
        text = tok.apply_chat_template(messages, tokenize=False,
            add_generation_prompt=True)
        inputs = tok(text, return_tensors="pt").to(model.device)
        with torch.inference_mode():
            out = model.generate(**inputs, max_new_tokens=256,
                do_sample=False, repetition_penalty=1.05)
        new = out[0, inputs.input_ids.shape[1]:]
        row.update({"model":a.model, "answer":tok.decode(new,
            skip_special_tokens=True)})
        f.write(json.dumps(row, ensure_ascii=False) + "
")
```

```
python scripts/generate_eval.py --model merged/sft \
  --out reports/sft_answers.jsonl
python scripts/generate_eval.py --model merged/dpo \
  --out reports/dpo_answers.jsonl
```

需要同时保存

模型 commit / 本地哈希、adapter 路径、数据集哈希、依赖 lock、CUDA / GPU 信息、训练参数、随机种子、评测 prompt 版本、生成参数和人工评审结果。

> **评测公平性** 比较 SFT 与 DPO 时先用 greedy decoding；若评估创作质量，再固定相同 temperature、top_p 与多次采样。

合并与 GGUF：从训练 checkpoint 变成部署文件

1. 合并 DPO adapter

```
python scripts/merge.py \
  --base merged/sft \
  --adapter runs/dpo/adapter_final \
  --out merged/dpo
```

2. 在训练机准备 llama.cpp

从文末官方 llama.cpp README 进入仓库，按 Build guide 获取源码。以下命令在仓库根目录执行：

```
cmake -B build -DCMAKE_BUILD_TYPE=Release -DLLAMA_CURL=ON
cmake --build build --config Release -j "$(nproc)" \
  --target llama-cli llama-server llama-quantize
```

3. 转成 F16 GGUF，再量化

```
mkdir -p ~/llm-lab/gguf
python convert_hf_to_gguf.py ~/llm-lab/merged/dpo \
  --outfile ~/llm-lab/gguf/model-f16.gguf --outtype f16

./build/bin/llama-quantize \
  ~/llm-lab/gguf/model-f16.gguf \
  ~/llm-lab/gguf/model-q4_k_m.gguf Q4_K_M
sha256sum ~/llm-lab/gguf/*.gguf > ~/llm-lab/gguf/SHA256SUMS
```

Q4_K_M

CPU VPS 的默认起点。文件小、速度和质量通常平衡较好。

Q5_K_M / Q8_0

有更多 RAM、对质量敏感时再比较；不要假设位数越高就一定值得。

转换位置 GGUF 转换和量化也建议在训练机完成。VPS 只接收最终 Q4/Q5 文件，可节省磁盘、内存和编译时间。

量化验收：上传前先在训练机跑通 API

```
./build/bin/llama-server \
  -m ~/llm-lab/gguf/model-q4_k_m.gguf \
  --host 127.0.0.1 --port 8080 \
  -c 2048 -np 1 --api-key-file ~/llm-lab/api_key.txt
```

```
curl http://127.0.0.1:8080/health
curl http://127.0.0.1:8080/v1/chat/completions \
  -H 'Content-Type: application/json' \
  -H "Authorization: Bearer $LLM_API_KEY" \
  -d '{
    "model":"local-domain-model",
    "messages":[{"role":"user","content":"E104 如何排查? "}],
    "temperature":0,
    "max_tokens":256
  }'
```

必须检查的四件事

1. **模板：**响应是否从助手角色开始，是否出现角色标签泄漏、循环或自问自答。
2. **一致性：**关键评测样本与 merged/dpo 的答案是否在可接受范围内。
3. **内存：**记录加载后 RSS、长上下文请求峰值和 OOM 行为。
4. **性能：**分别记录首 token 延迟、生成 tok/s；不要只看总请求时间。

上传到 VPS

```
rsync -avP ~/llm-lab/gguf/model-q4_k_m.gguf \
  ubuntu@VPS_IP:/opt/llm/models/
rsync -avP ~/llm-lab/gguf/SHA256SUMS \
  ubuntu@VPS_IP:/opt/llm/models/
```

将 `VPS_IP` 替换为实际地址。大文件传输后在 VPS 重新执行 `sha256sum`，确认未损坏。

GPU 部署：12GB 先选对权重格式

RTX 4070 上用 vLLM 的默认选择是 7B AWQ / GPTQ 4-bit；若要直接加载 bf16，则把模型控制在 3B 以下。模型“标称支持长上下文”不等于 12GB 能把上下文开到最大。

方案	12GB 适配度	何时用	第一轮参数
7B GPTQ 4-bit	推荐	已有发布好的 GPTQ checkpoint	ctx 4096；max-num-seqs 4
7B AWQ 4-bit	推荐	当前 vLLM 与 GPU 支持良好	ctx 4096；max-num-seqs 4
≤3B bf16	推荐	不想引入部署量化误差	ctx 4096；逐步加并发
7B fp16 / bf16	不可行	权重约 14GB，尚未计 KV cache	换量化或换更大显存
14B 4-bit	非默认	极短上下文、低并发的边界实验	容易被 KV / 运行时挤爆

你的微调模型如何进入 vLLM

1. 先把最终 DPO adapter 合并成标准 Hugging Face 权重。
2. 用与当前 vLLM 兼容的 AWQ / GPTQ 工具生成部署 checkpoint；保存量化配置和校准集哈希。
3. 在同一评测集上比较 merged/dpo 与量化权重。
4. 确认模型架构、量化 kernel 和 GPU compute capability 被当前 vLLM 版本支持，再上线。

不要混用格式 GGUF 主要面向 llama.cpp / Ollama；vLLM 的 12GB 默认路线是 AWQ / GPTQ checkpoint。不要把 CPU 部署量化文件当成 GPU 高吞吐服务的首选。

vLLM：RTX 4070 上的 OpenAI 兼容服务

安装与冻结版本

```
sudo mkdir -p /opt/vllm/{venv,secrets}
sudo chown -R "$USER:$USER" /opt/vllm
python3 -m venv /opt/vllm/venv
source /opt/vllm/venv/bin/activate
python -m pip install -U pip vllm openai
pip freeze > /opt/vllm/requirements.lock.txt
umask 077
openssl rand -hex 32 > /opt/vllm/secrets/api_key.txt
export VLLM_API_KEY="$(cat /opt/vllm/secrets/api_key.txt)"
```

7B GPTQ 4-bit 默认示例

```
source /opt/vllm/venv/bin/activate
export VLLM_API_KEY="$(cat /opt/vllm/secrets/api_key.txt)"
vllm serve Qwen/Qwen2.5-7B-Instruct-GPTQ-Int4   --host 127.0.0.1 --port 8000   --quantization gptq --dtype auto   --gpu-memory-utilization 0.88   --max-model-len 4096 --max-num-seqs 4   --served-model-name domain-gpu
```

```
curl http://127.0.0.1:8000/v1/chat/completions   -H "Authorization: Bearer $VLLM_API_KEY"   -H 'Content-Type: application/json'   -d '{"model":"domain-gpu","messages":[
    {"role":"user","content":"E104 如何排查? "}],
    "temperature":0,"max_tokens":256}'
```

OOM 时

- 先把 max-model-len 降到 2048
- max-num-seqs 降到 1
- gpu-memory-utilization 保持 0.85–0.90
- 检查是否有桌面 / 其它进程占显存

跑稳后

- 逐级增加并发与上下文
- 记录 TTFT、tok/s、峰值 VRAM
- 固定 vLLM、CUDA、驱动与模型 revision
- 配置 systemd 或容器重启策略

版本提示 vLLM 的量化支持和命令行参数会随版本变化。先按当前官方文档确认参数；首次启动成功后立即冻结依赖，不要在生产环境使用浮动 latest。

CPU VPS：llama.cpp Docker Compose 固化服务

目录与密钥

```
sudo mkdir -p /opt/llm/{models,secrets}
sudo chown -R "$USER:$USER" /opt/llm
cd /opt/llm
umask 077
openssl rand -hex 32 > secrets/api_key.txt
chmod 600 secrets/api_key.txt
sha256sum -c models/SHA256SUMS
```

compose.yaml

```
services:
  llama:
    image: ghcr.io/ggml-org/llama.cpp:server
    container_name: llama-server
    restart: unless-stopped
    ports:
      - "127.0.0.1:8080:8080"
    volumes:
      - ./models:/models:ro
      - ./secrets/api_key.txt:/run/secrets/api_key:ro
    command:
      - -m
      - /models/domain-v1-q4_k_m.gguf
      - --host
      - 0.0.0.0
      - --port
      - "8080"
      - --ctx-size
      - "2048"
      - --parallel
      - "1"
      - --threads
      - "4"
      - --api-key-file
      - /run/secrets/api_key
      - --no-webui
    mem_limit: 10g
    logging:
      options:
        max-size: "10m"
        max-file: "3"
```

```
cd /opt/llm
docker compose pull
docker compose up -d
docker compose logs -f --tail=100
docker stats llama-server
```

7B Q4 建议宿主机至少 12–16GB RAM；把 `--threads` 调成物理核心附近的值，并给操作系统留出内存余量。容器内监听 0.0.0.0，但宿主机端口只绑定 127.0.0.1，因此不会直接暴露公网。

CPU VPS：用 Ollama 简化 GGUF 管理

如果你更看重安装、模型命名和常驻服务的简单性，用 Ollama；如果你需要精细控制 slot、线程、KV cache 与 server 参数，用 llama.cpp。

1. 安装后导入自己的 GGUF

按 Ollama 官方 Linux 安装页完成安装。Ollama 导入 GGUF 时不会再次量化，所以应先用 llama.cpp 生成并验收 Q4_K_M。

```
cd /opt/llm/models
cat > Modelfile <<'EOF'
FROM /opt/llm/models/domain-v1-q4_k_m.gguf
PARAMETER num_ctx 2048
PARAMETER temperature 0
EOF
ollama create domain-v1 -f Modelfile
ollama run domain-v1 "E104 如何排查？"
```

2. 调用本地 API

```
curl http://127.0.0.1:11434/api/chat -d '{
  "model":"domain-v1",
  "messages":[{"role":"user","content":"E104 如何排查？"}],
  "stream":false
}'
```

3. 验收和运维

- 确认服务只在预期接口监听；公网访问仍走 SSH 隧道、VPN 或 TLS 反代。
- 记录 `ollama ps` 的加载状态与 CPU / RAM 峰值。
- 用版本化名称 `domain-v1`、`domain-v2`；新版本通过后再切客户端。
- 导入会写入 Ollama 模型存储；确认磁盘余量再保留多份大模型。

选 Ollama

个人使用、少量模型、想少写服务配置；接受较少的底层参数控制。

选 llama.cpp

需要固定镜像、明确资源限制、精细性能参数和直接 OpenAI 兼容端点。

模板仍要核对 Modelfile 的 FROM 可以读取 GGUF 中的聊天模板，但自训练模型仍必须用固定评测检查角色格式。必要时再显式提供 TEMPLATE，不能凭感觉抄别的模型模板。

访问方式：个人用 SSH 隧道，公网用 TLS

推荐：SSH 本地转发

不要直接开放推理端口。按实际后端选择一个本地转发：llama.cpp 8080、vLLM 8000、Ollama 11434。

```
# llama.cpp
ssh -N -L 8080:127.0.0.1:8080 ubuntu@VPS_IP
# vLLM 改成: -L 8000:127.0.0.1:8000
# Ollama 改成: -L 11434:127.0.0.1:11434
export OPENAI_BASE_URL=http://127.0.0.1:8080/v1
read -rsp "API key: " OPENAI_API_KEY; echo
export OPENAI_API_KEY
```

更稳妥的做法是通过密码管理器手动保存 API key；上面的第二行仅演示客户端变量，不要把密钥写进 shell history、代码仓库或日志。

客户端代码

```
from openai import OpenAI
import os
client = OpenAI(base_url=os.environ["OPENAI_BASE_URL"],
                api_key=os.environ["OPENAI_API_KEY"])
r = client.chat.completions.create(
    model="local-domain-model",
    messages=[{"role":"user","content":"E104 如何排查? "}],
    temperature=0, max_tokens=256)
print(r.choices[0].message.content)
```

必须公网提供时

- 域名指向 VPS，只开放 22、80、443；8080 继续只绑定本地。
- 用 Nginx / Caddy 终止 TLS，反代到 127.0.0.1:8080。
- 关闭代理缓冲以支持流式输出，读取超时设长一些。
- 保留 llama-server API key；再加限流、IP allowlist 或 VPN。
- 不要启用不受信任环境中的内置 agent / shell / 文件工具。

> **绝对不要** 把无鉴权的 0.0.0.0:8080 直接暴露公网。模型接口会被扫描、滥用并耗尽 CPU / 内存。

上线、回滚和容量：把模型当普通服务管理

版本化

```
/opt/llm/models/
├── domain-v1-q4_k_m.gguf
├── domain-v2-q4_k_m.gguf
├── SHA256SUMS
└── RELEASE_NOTES.md
```

compose 中不要永远写 `model.gguf`。明确指向版本文件；升级时先在 8081 做冒烟测试，再修改 compose，执行 `docker compose up -d`。失败就指回旧文件。

每日可见性

```
curl -fsS http://127.0.0.1:8080/health
docker inspect -f '{{.State.Status}} {{.RestartCount}}' llama-server
docker stats --no-stream llama-server
docker compose logs --since=1h --tail=200
```

容量保护

- 个人 VPS 从 parallel=1 开始
- 限制 max_tokens 与请求体
- 上下文从 2K 起，按需求上调
- 队列满时快速返回，而非无限排队
- 公网入口做速率限制

应备份什么

- 数据清洗脚本与哈希
- 每阶段 adapter
- 合并模型的 manifest
- 评测报告与失败样本
- 最终 GGUF + SHA256

GPU 与 CPU 如何长期分工

RTX 4070 在线且需要低延迟或多请求吞吐时，用 vLLM；显卡不常开机、服务要长期低成本运行时，用 CPU VPS 的 Ollama / llama.cpp。两条路线共用同一评测集和版本号，但部署权重格式不同。

> **性能现实** CPU 推理的体验受单核性能、内存带宽、虚拟化争用和峰时负载影响。先在你的 VPS 实测，再决定模型档位。

排错地图：症状先归因，再改参数

症状	最可能原因	先做什么
训练 CUDA OOM	序列 / batch / DPO 参考路径	MAX_LEN 1024→512，batch=1，累积 16
vLLM 启动 OOM	7B fp16、ctx / 并发过大	改 7B AWQ/GPTQ；ctx 2048；seqs=1
loss 为 NaN	精度、学习率、坏样本	改 bf16；降 LR；定位首个坏 batch
SFT 后胡言乱语	模板错、数据太少或过拟合	打印训练前渲染文本；与 base 对比
DPO 只会长回答	偏好数据长度偏置	做长度分桶，补“短而好”样本
CPT 后通用能力掉	灾难性遗忘	降 LR / epoch，混通用语料
GGUF 回答异常	chat template 或转换器版本	用同一 prompt 对比 merged 与 GGUF
VPS 启动即退出	RAM 不足、架构 / 镜像不匹配	看 logs；降模型 / ctx；确认 amd64/arm64
首 token 很慢	长 prompt、CPU 慢、冷缓存	缩短上下文；预热；记录 prompt tok/s
并发后崩溃	KV cache 与 slot 过大	parallel=1；降低 ctx；逐级压测

调参顺序

1. 先固定数据与模板，跑 20–50 step 冒烟。
2. 确认 loss 下降、checkpoint 可恢复、adapter 可合并。
3. 只改一个变量：LR、rank、epoch 或数据比例。
4. 用固定评测比较，不凭单个聊天印象。
5. 最后才扩模型、扩上下文、加并发。

版本漂移 TRL / Transformers 的参数名会变化。锁定成功环境；升级依赖时单独开分支，并重新执行最小冒烟与合并测试。

四周实战路线：从冒烟到双轨上线

四周最小可交付

第 1 周 · 数据与 CPT

做三类 JSONL、校验器、100 条冒烟；完成 CPT adapter、合并与领域 / 通用双评测。

第 2 周 · SFT

RTX 4070 上用 7B QLoRA：seq 1024、batch 1、累积 16；固定模板，完成格式评测。

第 3 周 · DPO

先标 100–500 对偏好；7B 从 seq 512 起步；检查长度偏置并做 SFT / DPO 盲评。

第 4 周 · 双轨部署

vLLM 验证 7B GPTQ/AWQ；GGUF Q4/Q5 对比；CPU VPS 压测、隧道、回滚演练。

完成标准

- 数据集可重复生成，原始来源、清洗脚本、切分与哈希齐全。
- CPT / SFT / DPO adapter、合并模型、配置和依赖 lock 可回滚。
- 12GB 上 7B QLoRA 冒烟、保存、恢复和合并均跑通。
- GPU：7B 4-bit vLLM；CPU：GGUF + llama.cpp 或 Ollama，至少各完成一次固定评测。
- 服务默认不暴露公网；有密钥、隧道 / TLS、容量限制、日志和回滚记录。

最终决策

你的优先级	选择	默认权重
低延迟、多请求、显卡常在线	vLLM	7B AWQ/GPTQ 4-bit
无需量化、模型很小	vLLM	≤3B bf16
低成本长期服务、低并发	llama.cpp	GGUF Q4_K_M
最省运维步骤	Ollama	导入已量化 GGUF

记住 12GB 上限 7B QLoRA 微调可行；7B 全参数微调不可行；vLLM 跑 7B fp16/bf16 会爆显存。先量化、短上下文、低并发，再逐步扩容。

官方与一手参考

1. https://huggingface.co/docs/trl/main/quickstart — TRL 的 SFT、DPO、GRPO 入口。
2. https://huggingface.co/docs/trl/v0.22.1/en/dpo_trainer — DPOTrainer 与偏好格式。
3. https://huggingface.co/docs/peft/main/developer_guides/lora — LoRA 配置与合并。
4. https://huggingface.co/docs/peft/main/en/developer_guides/quantization — 4-bit + LoRA 流程。
5. https://huggingface.co/docs/transformers/main/en/quantization/bitsandbytes — NF4、double quant 与 bitsandbytes。
6. https://docs.vllm.ai/en/latest/serving/online_serving/openai_compatible_server/ — vLLM 服务命令、鉴权和兼容 API。
7. https://docs.vllm.ai/en/latest/api/vllm/envs/ — VLLM_API_KEY 等环境变量。
8. https://huggingface.co/Qwen/Qwen2.5-7B-Instruct-GPTQ-Int4 — 7B GPTQ 4-bit 服务示例模型卡。
9. https://huggingface.co/Qwen/Qwen2.5-Coder-7B — 7B Base 训练示例模型卡。
10. https://github.com/ggerganov/llama.cpp/blob/master/README.md — 构建与 GGUF 入口。
11. https://github.com/ggml-org/llama.cpp/blob/master/tools%2Fserver%2FREADME.md — llama-server 参数与 API key。
12. https://github.com/ollama/ollama/blob/HEAD/docs/import.mdx — Ollama 导入 GGUF；导入时不再量化。
13. https://huggingface.co/Qwen/Qwen2.5-7B-Instruct-GGUF — 7B GGUF 量化类型参考。

超参数和显存档位是保守起点，不是质量或必然可运行的承诺。模型架构、驱动、CUDA、vLLM / TRL 版本、上下文和并发都会改变实际峰值；每次升级都应重新冒烟并记录显存。